Pygame - Time

25 November 2025 20:11

While using pygame, we sometimes needs to perform certain operations that include the uses of time. Like finding how much time our program has been running, pausing the program for an amount of time, etc. For operations of this kind we need to use the time method of pygame. The function are:

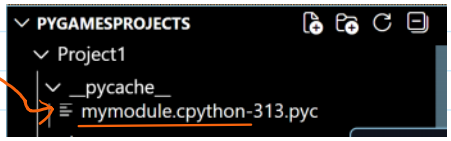
- pygame.time.wait
- pygame.time.get_ticks
- pygame.time.delay
- pygame.time.Clock

pygame.time.wait

This function is used to pause the running program for few seconds, it takes time in milliseconds as parameter. For example,

This method is present in this module

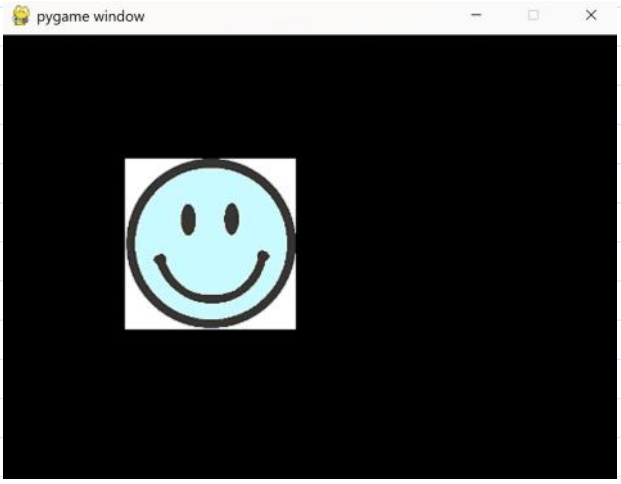
```
def quitFromApp():
    import pygame

    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            pygame.quit() # to quit pygame library
            quit() #to quit python execution environment.
```




```
import pygame
import mymodule
pygame.init()
display = pygame.display.set_mode((500,500))
image = pygame.image.load('Project1/images/image.png')
display.blit(image, (100, 100))
pygame.time.wait(5000)
while True:
    mymodule.quitFromApp()
    pygame.display.flip()
```

function is called from the module.

Output:



The output of this will be. That the script will wait for 5 seconds and then update the display to show the smiley logo.

It is slightly less accurate than pygame.time.delay .

pygame.time.get_ticks

This function gives a time which has been running in milliseconds. For example,

```
import pygame

pygame.init()
i = 0
while i < 5:
    ticks = pygame.time.get_ticks()
    print(ticks)
    i += 1
    pygame.time.wait(1000)
```

Output:

```
pygame 2.6.1 (SDL 2.28.4, Python 3.13.7)
Hello from the pygame community. https://www.pygame.org/contribute.html
691
1693
2694
3695
4696
```

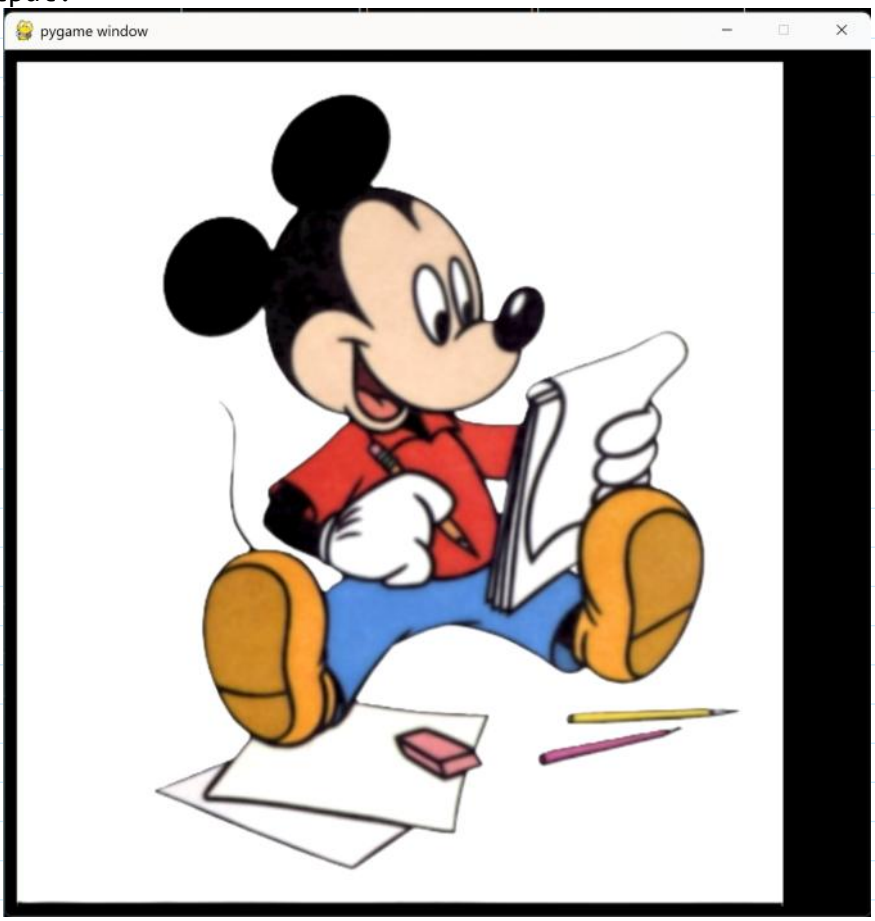
The time is printed for every iteration, including the time for which we paused the script in every iteration.

pygame.time.delay

This function works the same as pygame.time.wait function the difference is that this function will use the processor (rather than sleeping) in order to make the delay more accurate.

```
import pygame
import mymodule
pygame.init()
display = pygame.display.set_mode((700,700))
image = pygame.image.load("Project1/images/MickeyMouse.png")
display.blit(image, (100,100))
pygame.time.delay(5000)
while True:
    mymodule.quitFromApp()

    pygame.display.flip()
```

Output:



pygame.time.Clock

This function is used to create a clock object which can be used to keep track of time. The various method of Clock object are below:

tick(): This method should be called once per frame. It will compute how many milliseconds have passed since the previous call. If you pass the optional frame rate argument, the function will delay to keep the game running slower than the given ticks per second. For example, if we pass 10 as an argument, the program will never run at more than 10 frames per

second.

get_time(): It is used to obtain a number of milliseconds used between two tick()

get_fps(): It gives information regarding the clock frame rate. It returns the output in floating point value.

A simple program to demonstrate this function could be:

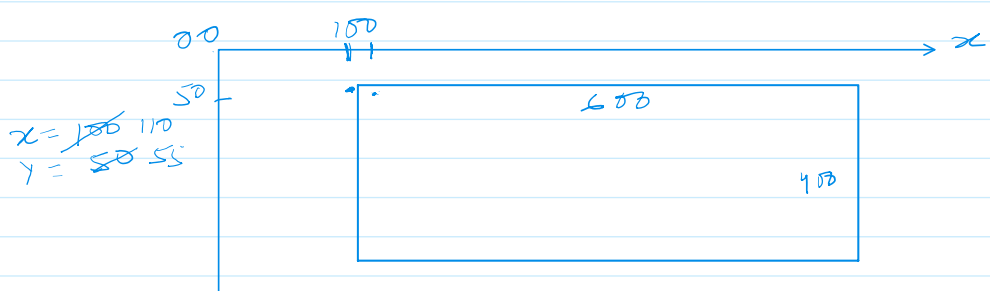
```
import pygame

pygame.init()
i = 0
clock = pygame.time.Clock() # Creating a Clock object.
while i < 5:
    # Setting FPS of a program to Max 1 per second.
    clock.tick(1)
    print(clock.get_time()) # Printing time used in the previous tick
    print(clock.get_fps()) # Printing compute the clock frame rate.
    i+=1
```

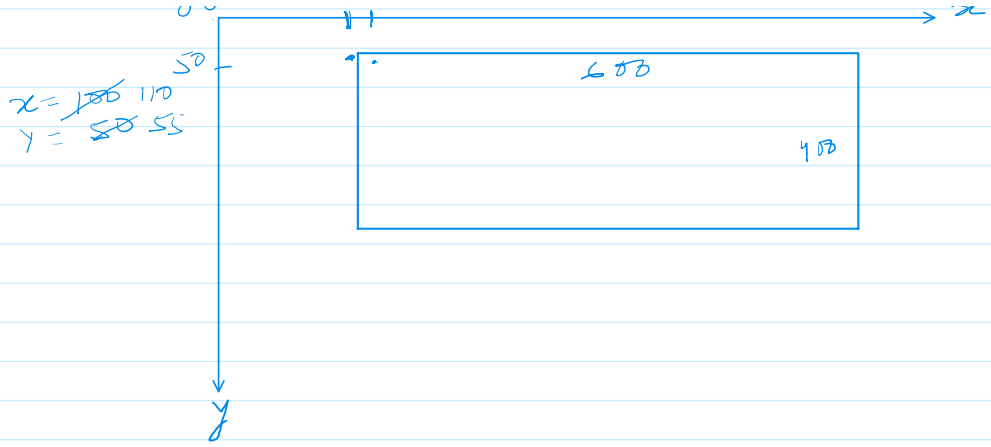
Output:

```
Hello from the pygame community. https://www.pygame.org/contribute.html
1002
0.0
1001
0.0
1002
0.0
1001
0.0
1001
0.0
```

As we have passed one in the tick method, it sets maximum FPS to one. Which results in time between each frame nearing 1000 milliseconds.

WAP to print 10 blue color rectangle inside each other, the rectangles will print with delay of 1 secs.





Home work:

1. Delayed Image Display
Write a Pygame program that loads two images. Display the first image for **3 seconds**, then automatically switch to the second image using **pygame.time.wait()**.

2. Digital Stopwatch Using get_ticks()
Create a Pygame window that shows a **running stopwatch** (MM:SS format) on the screen using **pygame.time.get_ticks()**.
The stopwatch should update every frame and stop when the user closes the window.

3. Frame Time Logger
Build a program that runs a loop for 10 iterations and prints:
- Current tick count using pygame.time.get_ticks()
- Time elapsed between frames using **clock.get_time()**
  Use a **Clock()** object to control the FPS to exactly *5 FPS*.

4. Smooth Animation With Time-Based Movement
Use **get_ticks()** to move a rectangle smoothly from the left to the right of the screen.
The movement must be based on *time passed*, not FPS (i.e., speed = 0.2 pixels per millisecond).

5. Accurate Delay Comparison
Write a program that prints timestamps before and after calling:
- pygame.time.wait(2000)
- pygame.time.delay(2000)
  Compare the difference and print which method is more accurate.

6. Countdown Timer
Display a **10-second countdown** on the Pygame window using **pygame.time.get_ticks()**.
When the timer reaches zero, show "TIME UP!" in red text.

7. Animation With tick() Frame Control
Create a bouncing ball animation. Use:
- clock.tick(60) to maintain 60 FPS
  Print the computed FPS using clock.get_fps() in the console every second.

8. Sequential Drawing With Delay
Draw **10 nested rectangles**, each 40 pixels smaller than the previous one.
Between drawing each rectangle, wait for **1000 ms** using **pygame.time.delay()**.

9. Measure Event Reaction Time
Display a blank screen for a random delay (1 to 4 seconds).
Then draw a circle.
The user must press SPACE as *fast as possible*.
Use pygame.time.get_ticks() to measure reaction time.

10. Multi-Timer System
Create three independent timers:

- A timer that updates text every **500 ms**
- A timer that changes background color every **1 second**
- A timer that prints a message in console every **2 seconds**

Use only **pygame.time.get_ticks()** (no sleep/wait).